ThermoBridge Project & Defence Guide

The recommended idea

Build exactly the unified engineering workbench requested by the assessment. A different impressive AI application would lose credit because the tasks, outputs and launch gate are fixed. ThermoBridge brings the required disciplines together through one small, inspectable C++ application.

What the package contains

Deliverable	Where to find it
Windows application	Launch.exe; workbench_cli.exe; core_tests.exe
Implementation	src/, CMakeLists.txt, tests/
Working examples	data/rod.stl and synthetic_temperature.csv
Numerical evidence	results/linux_run/ and results/plots/
Submission report	docs/Six_Page_Report.pdf, exactly six pages
Design and disclosure	Hardware_Design.md, AI_USE_LOG.md, DEPENDENCIES.md
Development record	development.bundle and COMMIT_LOG.txt
Your finishing tasks	Submission_Checklist.md and Demo_Script.md

Important status

The source and cross-compiled executable are provided. Numerical and concurrency checks were actually run on Linux. The Windows interface has not been executed on a real Windows 11 host here. The required clean-machine launch, editor run, handwritten scan and recorded demo remain your responsibility. Do not submit the ZIP without doing those checks.

Read this guide, inspect the corresponding functions, reproduce the results, and make one small tested change you understand. Disclosed AI use is permitted by the brief; pretending to have written or tested things you have not done is unnecessary and unsafe.

Understand the assessment

What earns marks

Criterion	Marks	Concrete evidence
Interface	8	One coherent app, readable plots, status and errors
C++ backend / tests	12	Real C++ processing and positive/negative/numerical tests
Architecture + readability	12	Separated core/UI, focused named functions
Version control	5	Real meaningful milestone history, not commit padding
Thermal FEA	14	Assembly, boundary conditions, midpoint error/convergence
CAD / ANSYS CSV	10	Both imports, camera, legend, inspection, validation
Multicore performance	10	Independent CPU work, fair timings and agreement
Embedded design	8	Interfaces/voltages/ownership plus three mocks
Backpropagation	7	Your handwritten derivation and C++ gradient checks
AI judgement	8	Transparent log, challenge tests and validation
Delivery / report	6	Standalone launch, report and reproducible evidence

The gates matter more than extra features

The assessment requires at least 6/12 in the C++ backend, 7/14 in thermal FEA and 3/6 in delivery/report, plus a working C++ backend and successful standalone launch. A beautiful interface does not compensate for a missing launcher. A high overall score does not bypass those gates.

Scope discipline

Do not add a chatbot, cloud database, 3D solid FEA or real FPGA programming before proving the required flows. Native ANSYS projects and CAD solids are explicitly outside scope. The synthetic CSV is acceptable when clearly labelled. The project is an engineering prototype, not a production solver.

Architecture you can explain

From button click to a reliable result

A user selects a view and requests an operation. The UI reads parameters and validates simple syntax. The C++ core checks physical bounds, parses or solves, and returns data structures. The UI draws the result. A background thread owns temporary work so the window can keep handling input.

Type / function	Meaning
Rod	Element count, length, area, conductivity, generation and endpoint temperatures
Solution	Node coordinates, nodal temperatures and maximum midpoint error
solve(Rod)	Validated FE assembly and tridiagonal solution
parse_stl / parse_csv	Strict data conversion with explicit input limits
generate_cases / run_batch	Reproducible cases and serial/parallel independent computation
benchmark	Warm-up, repeated compute timing, agreement and summary
Device	A mock connection state, capability description and telemetry sequence

Why no web server?

A native interface removes the need to bundle a browser runtime or a local HTTP service. The entire app can run offline with standard Windows components. The cost is a simpler visual style, Windows-only presentation code and display/accessibility limitations that must be tested.

Why cancellation is safe

Workers write into a new result object. They do not modify the displayed result while computing. Batch cancellation is checked between cases; all threads are joined before the operation reports cancellation. The UI replaces displayed data only after successful completion. A single imported file or rod solve finishes its current unit safely, then its result can be discarded.

What to demonstrate in the code

Open thermal.cpp at solve(), experiments.cpp at run_batch(), and winmain.cpp at begin() and WM_APP+1. Explain who owns each result and why no renderer reads a vector while a worker is resizing it.

Thermal FEA from first principles

Physical meaning

Imagine a thin metal rod. The left end is held at 100 C and the right at 20 C. Heat is generated throughout its volume, and no heat leaves through the sides. Conductivity tells you how easily heat travels along the rod. Steady state means temperature is no longer changing with time.

Units and governing equation

k has units W/(m K); A is m²; dT/dx is K/m. The conducted heat rate -kA*dT/dx is therefore watts. The generation per unit length qA is W/m. Energy balance gives -d(kA*dT/dx)/dx=qA. Temperature differences in Celsius and Kelvin are equal, so Celsius endpoint values are valid here.

Why the curve rises above both endpoint temperatures

Internal generation adds heat inside the rod. The maximum occurs where dT/dx=0. For this default case dT/dx=420-1000x, so the exact peak is at x=0.42 m and T=188.2 C. A linear interpolation between the boundary temperatures alone would ignore all internally generated heat.

Element approximation

Split the rod into N equal intervals of length h=L/N. On each interval approximate temperature as N1*T1+N2*T2, where N1=1-xi/h and N2=xi/h in local coordinates. Each shape function is one at its own node and zero at the other node. Its derivative is constant, giving a constant gradient in each element.

Integrating B-transpose*kA*B over an element gives K_e=(kA/h)[[1,-1],[-1,1]]. Integrating shape functions times qA gives f_e=(qAh/2)[1,1]. Both nodes receive half the element generation. The shared node receives contributions from both neighboring elements.

Why the area cancels from the exact temperature

With uniform area, both conduction and volumetric generation contain A. Dividing the differential equation by A removes it. The implementation still assembles A correctly; it would matter if heat input were specified differently or the area varied.

Assembly, boundaries and error

A concrete two-element example

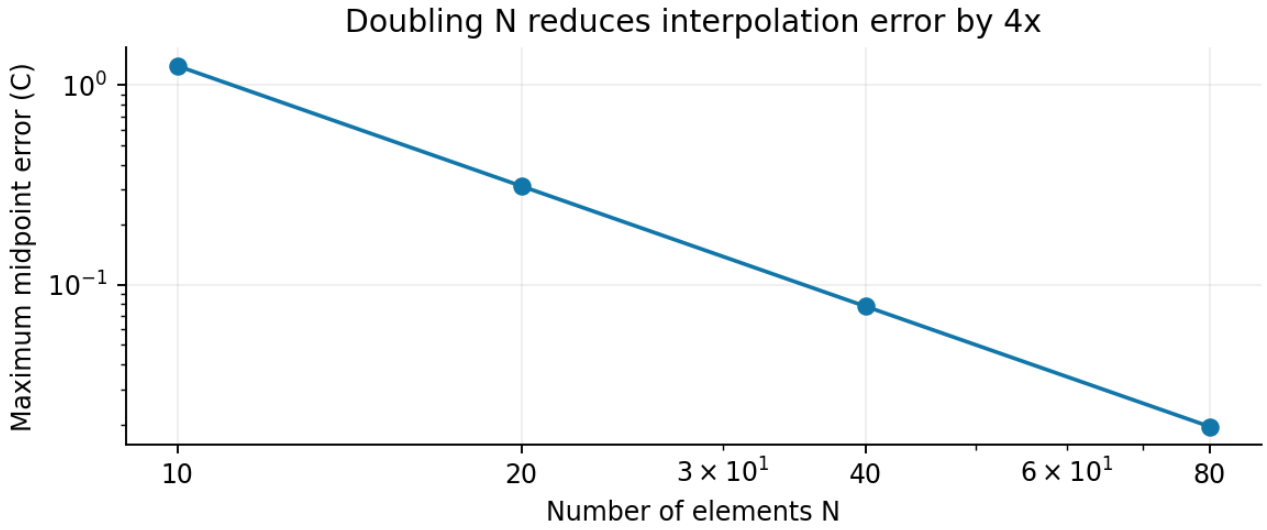
For N=2, h=0.5 m, kA/h=0.002 W/K and each element load at each endpoint is 0.25 W. The interior node receives 0.5 W. Its equation is -0.002*100+0.004*T1-0.002*20=0.5. Therefore T1=185 C. This small case is an independent unit test of assembly and prescribed-temperature handling.

Impose both boundaries correctly

The unknown interior equations contain coefficients multiplying the known endpoint temperatures. Move those terms to the right-hand side first. Then remove the associated couplings and replace endpoint rows with identity rows. Simply overwriting the endpoint RHS without correcting adjacent equations is a common error.

Why Thomas rather than a matrix inverse

Each node interacts only with its neighbors, so the global matrix is tridiagonal. Store the lower diagonal, main diagonal and upper diagonal. Forward elimination followed by back substitution is O(N). A dense N-by-N matrix wastes memory, and explicitly computing its inverse is unnecessary.



The exact midpoint-error argument

The exact temperature has second derivative -q/k. Linear interpolation of a quadratic differs most at the element midpoint. The magnitude is |T"|h²/8=q*h²/(8k). Halving h divides the error by four; log(error_old/error_new)/log(2)=2. Nodal agreement alone does not validate interpolation accuracy.

Default values: 10 elements -> 1.25 C; 20 -> 0.3125 C; 40 -> 0.078125 C; 80 -> 0.01953125 C. Find these values in convergence.csv and explain where they come from without looking at the code.

Geometry and CSV workflow

Two independent datasets

The STL viewer displays surface triangles. The CSV viewer displays discrete positions with a temperature attached to each node. The application does not claim a registration between those datasets. The separate thermal solver models an idealized 1D rod; it does not solve the imported STL mesh.

Sample files

rod.stl is a 1 m x 0.01 m x 0.01 m rectangular prism with twelve triangles. synthetic_temperature.csv has four cross-section corner nodes at each of 21 positions. Temperatures follow the stated analytical equation. It is synthetic data in the requested export format, not a real ANSYS run.

CSV column	Meaning
node_id	Unique nonnegative integer identifier
x_m, y_m, z_m	Cartesian coordinates in meters
temperature_C	Finite temperature in Celsius

Viewer controls

Use left-drag to rotate the orthographic projection. Right-drag changes the screen offset. The mouse wheel changes zoom. Fit-to-view resets rotation/zoom/pan and scales the bounding box to the viewport. In point-cloud mode, clicking within a small screen distance selects the nearest node and reports its ID, coordinates and temperature.

Validation you should be able to defend

A strict parser refuses empty files, missing fields, repeated IDs, NaN/Inf and numeric tokens with trailing junk. The STL grammar requires exactly three vertices per facet and a closing endsolid. A zero-area triangle is rejected. Limits prevent unbounded memory use. These are explicit supported-format decisions, not claims to handle every CAD exporter.

Useful small change for ownership

Add a clear selected-node highlight or display the input filename in the geometry view. Keep the core format unchanged, test a valid and invalid import, and commit the change with a specific message. This is an honest contribution you can demonstrate live.

Parallel performance explained

Concurrency is not automatically parallel computation

Putting one solve on a background UI thread keeps the interface responsive, but does not demonstrate the required multicore batch. This implementation has a separate bounded group of CPU workers, each solving independent rods. The UI background job merely coordinates that benchmark.

Determinism

Generate all cases once from seed 2027 before timing. Each worker gets an input index with an atomic fetch-add. Results are written to the preallocated slot with that same index. Completion order can vary, but output order does not. Every solve uses the same numerical operations, so comparison can be exact in this implementation.

Workers	Median (s)	Speedup	Efficiency	Max delta (C)
1	0.078966	1.000x	100.0%	0
2	0.043431	1.818x	90.9%	0
4	0.024241	3.258x	81.4%	0

Interpret the measurements

For this run, T1 is about 0.0790 s and T4 about 0.0242 s, so speedup is about 3.26 and efficiency about 81.4%. These are measurements of this specific shared Linux environment and workload. They are not a promise for the candidate laptop or Windows build.

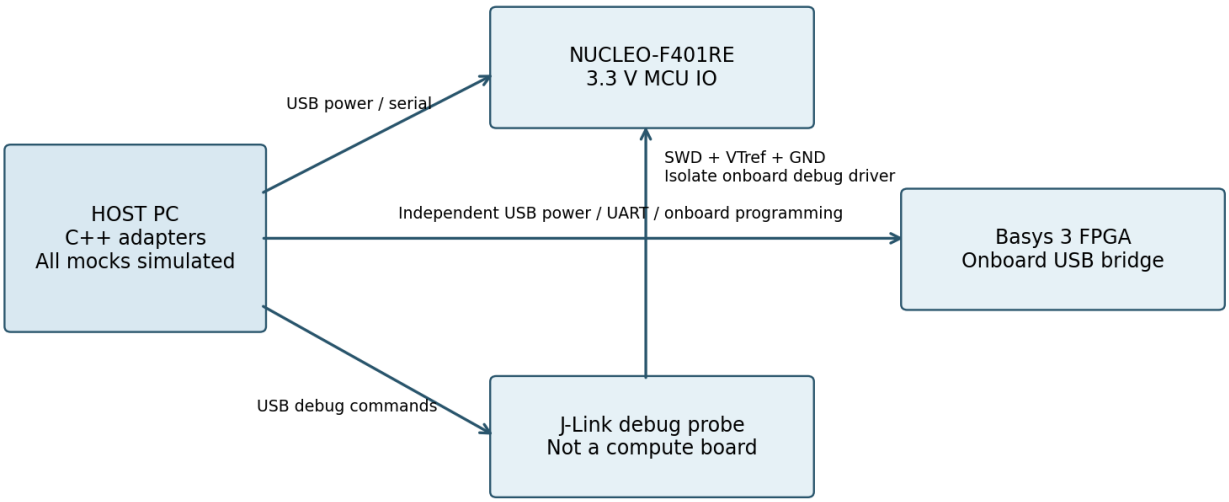
Why report five repetitions and a median?

One run can be distorted by scheduling, other workloads or cache conditions. Warm-up reduces first-use effects. Five raw samples make the summary inspectable. The median is less sensitive to a single slow sample, but five runs are still a small sample and do not establish a full statistical model.

Limits and fair timing

Timing includes thread creation/joining and allocations because users pay those costs. Generation, comparison, loading, drawing and compilation are excluded. For small workloads startup overhead can exceed the benefit. For large workloads memory bandwidth and shared-host scheduling can dominate. A single-logical-CPU host uses the same worker function but cannot demonstrate multicore speedup.

Hardware design you can defend



Design-only connections. Verify exact board/probe revisions before wiring. No direct MCU-FPGA cable required.

Three different responsibilities

The microcontroller runs sequential firmware and exposes simple control/telemetry. The FPGA implements configurable logic that can operate concurrently. The JTAG/SWD adapter carries debug commands between the host and a supported target. It does not replace either compute board.

Electrical assumptions are not interchangeable

USB power and GPIO signal levels are different. A board may accept USB power while its chip IO uses a lower voltage. The reference design uses the board regulators and does not connect a 5 V signal to 3.3 V IO. Common ground is required for a direct debug signal path. VTref senses target voltage; it is not an instruction to power the target from the probe.

One driver per debug interface

If an onboard ST-LINK and an external J-Link both drive the same debug signal, they can contend. The onboard debug path must be isolated before an external probe becomes the owner. JTAG has TCK, TMS, TDI and TDO; SWD uses a clock and bidirectional data. This design uses SWD for the MCU and the FPGA board's own programming bridge.

The mock failure is deliberately recoverable

Connect sets the simulated device state. Read telemetry returns a labelled message and sequence number. Inject timeout causes exactly the next read to fail, clears the injected flag, and preserves the connected state. Retrying succeeds. This tests application recovery logic without claiming to have measured a physical timeout.

The detailed pin/voltage table and official manual links are in Hardware_Design.md. Verify actual hardware revisions before wiring. Real drivers, firmware, FPGA programming and physical measurements are outside this prototype.

Handwriting reference: page 1

Write this derivation yourself; this typed page is not the appendix

Given: x=1, y=2, w=0.5, b=0, v=1, c=0, learning rate alpha=0.1.

1. Forward pass

z=wx+b=0.5*1+0=0.5
h=tanh(z)=0.4621171573
prediction=vh+c=1*0.4621171573+0=0.4621171573
residual e=prediction-y=-1.5378828427
J=0.5*e²=1.1825418190

2. Local derivatives

dJ/dprediction=e=-1.5378828427
dprediction/dv=h=0.4621171573
dprediction/dc=1
dprediction/dh=v=1
dh/dz=1-tanh²(z)=1-h²=0.7864477330
dz/dw=x=1
dz/db=1

3. Chain rule for all four parameters

dJ/dv=(dJ/dprediction)*(dprediction/dv)=e*h
= -0.7106820475

dJ/dc=(dJ/dprediction)*(dprediction/dc)=e
= -1.5378828427

dJ/dw=e*v*(1-h²)*x
= -1.2094644752

dJ/db=e*v*(1-h²)
= -1.2094644752

Explain the matching gradients

The w and b gradients are equal for this sample because x=1. In general dJ/dw includes x while dJ/db does not. Their equality here is not a rule for every regression sample.

Write intermediate values clearly and retain enough digits to avoid inconsistent rounded results. Use arrows to show the path J -> prediction -> h -> z -> w/b. A correct derivation matters more than decorative handwriting.

Handwriting reference: page 2

4. Simultaneous gradient-descent update

Compute every gradient from the original parameters. Then update all four together using theta_new=theta_old-alpha*gradient.

Parameter	Update	New value
w	0.5 - 0.1*(-1.2094644752)	0.6209464475
b	0 - 0.1*(-1.2094644752)	0.1209464475
v	1 - 0.1*(-0.7106820475)	1.0710682047
c	0 - 0.1*(-1.5378828427)	0.1537882843

5. Explain gradient direction

All initial gradients are negative. Increasing each parameter a small amount locally reduces the loss because the gradient points toward increasing loss and we move against it. The model initially underpredicts the target. The updated loss is 0.6857737650, below 1.1825418190. This does not imply every learning rate will decrease loss.

6. Numerical verification in C++

For parameter theta, numerical gradient = [J(theta+epsilon)-J(theta-epsilon)]/(2*epsilon), epsilon=1e-6. Keep the other three parameters fixed at their original values. Compare the analytical and numerical numbers using absolute error.

Parameter	Absolute error
w	1.09025677375e-10
b	1.09025677375e-10
v	5.25383070382e-11
c	1.11754827614e-10

7. Training result

Step 0 is the initial model. Step 1 is the handwritten update. Steps 2 through 101 are the required 100 subsequent updates at the same learning rate. The loss falls to approximately 1.08e-20. The system fits one sample well; it has not been evaluated on unseen samples and cannot be called a general regression solution.

Submit at most two legible handwritten pages with your own explanation. Do not label this typed reference as handwritten work, and do not sign an understanding statement until you can reconstruct the chain rule yourself.

Windows verification & demo

Run from the extracted package

Extract everything into a writable folder. Double-click Launch.exe. Confirm that it opens without installing tools, building source or using the internet. Keep data/ beside the executable so sample buttons can find their files. Write down the Windows build and screen scaling used.

Check	Expected result
Geometry sample	12-triangle wireframe; rotate/pan/zoom/fit work
CSV sample	84 coloured nodes with Celsius legend and inspected node details
Invalid CSV	Actionable non-finite-data error; old valid cloud retained
Thermal defaults	Endpoints 100/20 C; curve peaks near 188.2 C
Convergence	Midpoint errors reduce by factor 4
Benchmark	1/2/up to 4 workers; 5 times each; max delta <=1e-8 C
Cancellation	UI remains usable; completed old result preserved
Hardware	All three mocks; one-shot timeout; retry succeeds
Backprop	Four gradient errors below 1e-5; loss plot and CSV
Editor	Selected run configuration opens the same packaged app

Measure launch honestly

runtime_ready_* logs measure from program entry to the first ready timer and include its 200 ms delay. They exclude OS loader time. Use a screen recording or stopwatch for double-click-to-ready. Keep that separate from compute timings.

The required 3-5 minute demonstration

Follow Demo_Script.md: launch, imports and validation, thermal verification, benchmark, hardware failure/recovery, gradient check and disclosure. If using screenshots instead, capture actual UI states with timestamps and matching exported logs. The package does not contain a fabricated recording or a mock screenshot passed off as evidence.

Defence questions & next steps

Likely question	A defensible short answer
Why is this FEA?	We assemble element stiffness/load contributions using linear shape functions, then solve nodal unknowns.
Why midpoint error?	Nodal values can be nearly exact here; interpolation between nodes still has $O(h^2)$ error.
Why no matrix inverse?	The system is tridiagonal; Thomas solves it in linear time and storage.
Why two thread layers?	One keeps the UI responsive; bounded CPU workers perform independent parallel solves.
Why deterministic order?	Each worker writes to the slot belonging to its original case index.
Why is speedup below 4?	Thread startup, allocation, memory traffic, scheduling and serial overhead cost time.
Is this a real ANSYS export?	No. It is explicitly synthetic data matching the required nodal-export schema.
Can J-Link replace an FPGA?	No. A debug probe communicates with a supported target; it is not a configurable compute fabric.
Why update gradients together?	Recomputing some gradients after earlier updates changes the requested gradient-descent step.
What did AI do?	It generated the baseline code and documents; my personal edits and checks are recorded separately.
What is not verified?	Native Windows launch/editor/display behavior, real hardware, and the candidate-written appendix until I complete them.

Suggested completion order

1. Verify Launch.exe on Windows before polishing anything else.
2. Reproduce solver, gradient and benchmark outputs.
3. Study and handwrite the two-page derivation.
4. Make one useful change you understand and add tests where needed.
5. Record the demonstration and actual effort.
6. Update ownership/disclosure/report status, refresh the Git bundle, rename the ZIP with your candidate ID.

Do not claim a predicted score or guaranteed shortlist. The assessors evaluate the actual submitted behavior and may ask you to modify code live. A small project you can explain and verify is stronger than additional features you cannot defend.